

Experiment No. 7

AIM : Implement any one of the Local Search Techniques.

Theory :

Definition of Stimulated Annealing Algorithm –

Simulated Annealing is an optimization algorithm designed to search for an optimal or near-optimal solution in a large solution space. The name and concept are derived from the process of annealing in metallurgy, where a material is heated and then slowly cooled to remove defects and achieve a stable crystalline structure. In Simulated Annealing, the "heat" corresponds to the degree of randomness in the search process, which decreases over time (cooling schedule) to refine the solution. The method is widely used in combinatorial optimization, where problems often have numerous local optima that standard techniques like gradient descent might get stuck in. Simulated Annealing excels in escaping these local minima by introducing controlled randomness in its search, allowing for a more thorough exploration of the solution space.

Advantages of Simulated Annealing

- Simple Implementation
- Ability to Escape Local Minima
- Global Optimization
- Flexibility

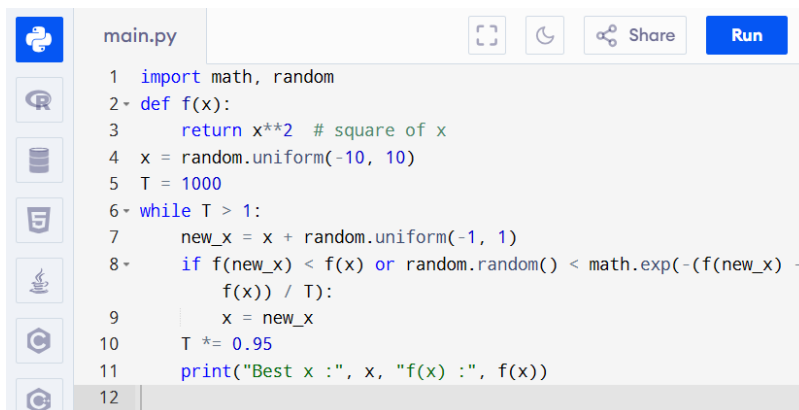
Limitations of Simulated Annealing

- Parameter Sensitivity
- Computational Time
- Slow Convergence

Algorithm for implementing Simulated Annealing :

- S1] Start with the initial random solution.
- S2] Evaluate its cost.
- S3] Generate neighbor solution.
- S4] If neighbor better -> accept.
- S5] If neighbor is worst -> accept with probability.
 $P = -\Delta E/T$
 ΔE = Increase in cost
 T = Temperature
- S6] As temperature get down choosing of worst case scenario is also less.
- S7] Stop.

Code :



```
main.py
1 import math, random
2 def f(x):
3     return x**2 # square of x
4 x = random.uniform(-10, 10)
5 T = 1000
6 while T > 1:
7     new_x = x + random.uniform(-1, 1)
8     if f(new_x) < f(x) or random.random() < math.exp(-(f(new_x) -
9         f(x)) / T):
10         x = new_x
11     T *= 0.95
12     print("Best x :", x, "f(x) :", f(x))
```

Output :

Output	Clear	Output	Clear
Best x : -2.7853680034827293 f(x) : 7.758274914825366		Best x : -0.28197287847214314 f(x) : 0.079508704193866	
Best x : -2.280530324704464 f(x) : 5.200818561896647		Best x : -0.43717403397652643 f(x) : 0.1911211359833091	
Best x : -1.5793911522376511 f(x) : 2.494476411766575		Best x : 0.0013745350095344389 f(x) : 1.88934649243584e-06	
Best x : -1.9156699755860067 f(x) : 3.6697914553616915		Best x : 0.03767826737451263 f(x) : 0.0014196518323452629	
Best x : -1.3153571346133082 f(x) : 1.7301643915781324		Best x : 0.10464886783768423 f(x) : 0.010951385539709101	
Best x : -0.45346176097314794 f(x) : 0.20562756866486837		Best x : 0.9320623998906741 f(x) : 0.8687403172899628	
Best x : -0.9080160517432911 f(x) : 0.8244931502234751		Best x : 0.554901416333506 f(x) : 0.3079155818489309	
Best x : -1.0804645227780307 f(x) : 1.1674035849819577		Best x : 0.5118824826616482 f(x) : 0.26202367605585253	
Best x : -0.8827162323995423 f(x) : 0.7791879469416428		Best x : 0.43377580492356027 f(x) : 0.18816144893708261	
Best x : -0.3292885346978536 f(x) : 0.10843093908345952		Best x : 0.3680323711818845 f(x) : 0.1354478262377604	
Best x : -0.015031108879508759 f(x) : 0.00022593423414764705		Best x : 0.9247629035501612 f(x) : 0.8551864277825247	
Best x : 0.4081815213897768 f(x) : 0.16661215440407282		Best x : 0.1548752954564223 f(x) : 0.023986357142714104	
Best x : -0.17850861032173726 f(x) : 0.03186532395899784		Best x : -0.28623738857442027 f(x) : 0.08193184261790366	
Best x : -0.10236365469315034 f(x) : 0.01047831780213852		Best x : -0.38246126132390135 f(x) : 0.14627661641346956	
Best x : -1.0451810089195963 f(x) : 1.0924033414061853		Best x : -0.515409546596709 f(x) : 0.26564700072302516	
Best x : -1.4696205169342251 f(x) : 2.159784463794019		Best x : -0.8450216293418515 f(x) : 0.7140615540555575	
Best x : -0.6960323793481207 f(x) : 0.48446107310100617		Best x : 0.0015361444886468067 f(x) : 2.3597398899999595e-06	
Best x : 0.29944400742556954 f(x) : 0.08966671358308455		Best x : -0.23060201514255452 f(x) : 0.05317728938780694	
Best x : -0.5867876127202267 f(x) : 0.3443197024419028		Best x : -0.2935889096568296 f(x) : 0.08619444787348605	
Best x : -1.5263385951816812 f(x) : 2.3297095071411884		Best x : 0.4250017147262686 f(x) : 0.18062645752026857	
Best x : -0.9677434666348854 f(x) : 0.9365274172145056		Best x : 0.26516734708106116 f(x) : 0.07031372195800795	
Best x : -0.5547995271120141 f(x) : 0.30780251528371444		Best x : 0.21046869903325116 f(x) : 0.04429707327274926	
Best x : -1.1682897726828192 f(x) : 1.3649009929552733		Best x : 0.4255761887671614 f(x) : 0.1811150924455826	
Best x : -1.059042563587993 f(x) : 1.1215711361789806		Best x : -0.07315292065486312 f(x) : 0.005351349800336699	
Best x : -0.16182839511039182 f(x) : 0.026188429464005087		Best x : 0.6555223330004356 f(x) : 0.429709529062334	
Best x : -0.9898553606613172 f(x) : 0.9798136350299463		Best x : -0.2627514994184421 f(x) : 0.06903835044663958	

Conclusion :

In this experiment, we implemented the Simulated Annealing algorithm and observed its ability to escape local minima and move toward near-optimal solutions. The results show that, with proper parameter tuning, it is a simple yet effective method for solving complex optimization problems.